

Informe de Trabajo Práctico 4

Aprendizaje Profundo con aplicación a Visión Artificial

Laura Zúñiga Osorio

17 de noviembre de 2025

1. Cálculo de DKL

En este ejercicio probamos algunas probabilidades de la Divergencia de Kullback-Leibler, definida como:

$$D_{KL}(p||q) = \int p(x) \log \frac{p(x)}{q(x)} dx . \quad (1)$$

No negatividad. Usamos el hecho de que $\log t \leq t - 1 \ \forall \ t > 0$. Tomamos $t = \frac{q(x)}{p(x)}$ (con $p(x) > 0$). Entonces

$$\begin{aligned} D_{KL}(p||q) &= \int p(x) \log \frac{p(x)}{q(x)} dx = - \int p(x) \log \frac{q(x)}{p(x)} dx \\ &\geq - \int p(x) \left(\frac{q(x)}{p(x)} - 1 \right) dx = - \int q(x) dx + \int p(x) dx = -1 + 1 = 0. \end{aligned}$$

Por tanto, $D_{KL}(p||q) \geq 0$.

Deducción de la cota variacional. En primer lugar, sabemos que la integral de una distribución de probabilidad es 1. Por tanto,

$$\log p(x) = \log p(x) \cdot 1 = \log p(x) \int q(z|x) dz = \int q(z|x) \log p(x) dz. \quad (2)$$

La regla de Bayes nos dice que $p(x, z) = p(x)p(z|x)$, entonces $\log p(x) = \log \frac{p(x, z)}{p(z|x)}$. Escribimos

$$\log p(x) = \int q(z|x) \log p(x) dz = \int q(z|x) \log \frac{p(x, z)}{p(z|x)} dz \quad (3)$$

Dividiendo arriba y abajo por $q(z|x)$ dentro del logaritmo:

$$\log p(x) = \int q(z|x) \log \frac{p(x, z)q(z|x)}{p(z|x)q(z|x)} dz \quad (4)$$

$$= \int q(z|x) \left[\log \frac{q(z|x)}{p(z|x)} + \log \frac{p(x, z)}{q(z|x)} \right] dz \quad (5)$$

$$= \int q(z|x) \log \frac{q(z|x)}{p(z|x)} dz + \int q(z|x) \log \frac{p(x, z)}{q(z|x)} dz \quad (6)$$

$$= D_{KL}(q(\cdot|x) \parallel p(\cdot|x)) + \int q(z|x) \log \frac{p(x, z)}{q(z|x)} dz \quad (7)$$

Obtención de error de reconstrucción + $\hat{\mathbf{D}}_{\mathbf{KL}}$. Partimos del último término de la eq. (7), reescribiéndolo (por Bayes) como

$$B = \int q(z|x) \log \frac{p(z)p(x|z)}{q(z|x)} dz \quad (8)$$

$$= \left[\int q(z|x) \log p(x|z) - \log \frac{q(z|x)}{p(z)} \right] dz \quad (9)$$

$$= \left[\int q(z|x) \log p(x|z) dz \right] - \left[\int q(z|x) \log \frac{q(z|x)}{p(z)} dz \right] \quad (10)$$

$$= \left[\int q(z|x) \log p(x|z) dz \right] - D_{KL}(q(\cdot|x) \parallel p(\cdot)) \quad (11)$$

Cálculo para distribución Gaussiana. Si $p = \mathcal{N}(\mu_p, \sigma_p^2)$ y $q = \mathcal{N}(\mu_q, \sigma_q^2)$, entonces

$$D_{KL}(p \parallel q) = \int p(x) \log \frac{p(x)}{q(x)} dx \quad (12)$$

$$= \int p(x) \left[\log \frac{\sigma_q}{\sigma_p} + \frac{(x - \mu_q)^2}{2\sigma_q^2} - \frac{(x - \mu_p)^2}{2\sigma_p^2} \right] dx \quad (13)$$

$$= \log \frac{\sigma_q}{\sigma_p} + \frac{1}{2\sigma_q^2} \mathbb{E}_p[(x - \mu_q)^2] - \frac{1}{2\sigma_p^2} \mathbb{E}_p[(x - \mu_p)^2] \quad (14)$$

$$= \log \frac{\sigma_q}{\sigma_p} + \frac{\sigma_p^2 + (\mu_p - \mu_q)^2}{2\sigma_q^2} - \frac{1}{2} \quad (15)$$

$$= \log \frac{\sigma_q}{\sigma_p} + \frac{\sigma_p^2 + (\mu_p - \mu_q)^2}{2\sigma_q^2} - \frac{1}{2}. \quad (16)$$

2. Auto-encoder variacional

En este ejercicio entrenamos y evaluamos la performance de un autoencoder variacional convolucional que aprende a generar dígitos manuscritos a partir de la base de datos MNIST. Para ello, construimos una red del tipo encoder-decoder con la siguiente arquitectura:

Model: "encoder"

Layer (type)	Output Shape	Param #	Connected to
input_layer_1 (InputLayer)	(None, 28, 28, 1)	0	-
conv2d_2 (Conv2D)	(None, 28, 28, 32)	320	input_layer_1[0]...
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 32)	0	conv2d_2[0][0]
conv2d_3 (Conv2D)	(None, 14, 14, 64)	18,496	max_pooling2d_2[...]
max_pooling2d_3 (MaxPooling2D)	(None, 7, 7, 64)	0	conv2d_3[0][0]
flatten_1 (Flatten)	(None, 3136)	0	max_pooling2d_3[...]
dense_3 (Dense)	(None, 16)	50,192	flatten_1[0][0]
dense_4 (Dense)	(None, 16)	50,192	flatten_1[0][0]
reparametrize_laye... (ReparametrizeLaye...)	(None, 16)	0	dense_3[0][0], dense_4[0][0]

Model: "decoder"

Layer (type)	Output Shape	Param #
decoder_input (InputLayer)	(None, 16)	0
dense_5 (Dense)	(None, 3136)	53,312
reshape_1 (Reshape)	(None, 7, 7, 64)	0
up_sampling2d_2 (UpSampling2D)	(None, 14, 14, 64)	0
conv2d_transpose_2 (Conv2DTranspose)	(None, 14, 14, 32)	18,464
up_sampling2d_3 (UpSampling2D)	(None, 28, 28, 32)	0
conv2d_transpose_3 (Conv2DTranspose)	(None, 28, 28, 1)	289

Model: "vae"

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 28, 28, 1)	0
encoder (Functional)	[(None, 16), (None, 16), (None, 16)]	119,200
decoder (Functional)	(None, 28, 28, 1)	72,065

Total params: 191,265 (747.13 KB)
Trainable params: 191,265 (747.13 KB)
Non-trainable params: 0 (0.00 B)

Figura 1: Arquitectura de autoencoder variacional convolucional, separando en redes distintas al encoder y el decoder.

La capa de reparametrización se encarga de muestrear puntos en el espacio latente para servir como entradas al decoder. Está definida de la forma:

```

1 class ReparametrizeLayer(keras.Layer):
2 def call(self, mu, logvar, training=None):
3     self.mu = mu
4     self.logvar = logvar
5     std = tf.exp(0.5 * logvar)
6     if training:
7         eps = tf.random.normal(shape=tf.shape(std))
8         return mu + eps * std
9     else:
10        return mu

```

La red se entrenó por 100 épocas usando como función de pérdida el error de reconstrucción y la Divergencia de Kullback Leibler entre las distribuciones de entrada (conjunto de entrenamiento) y de salida (predicciones). Los dígitos generados a partir de 10 puntos sampleados del ReparametrizeLayer se muestran a continuación:

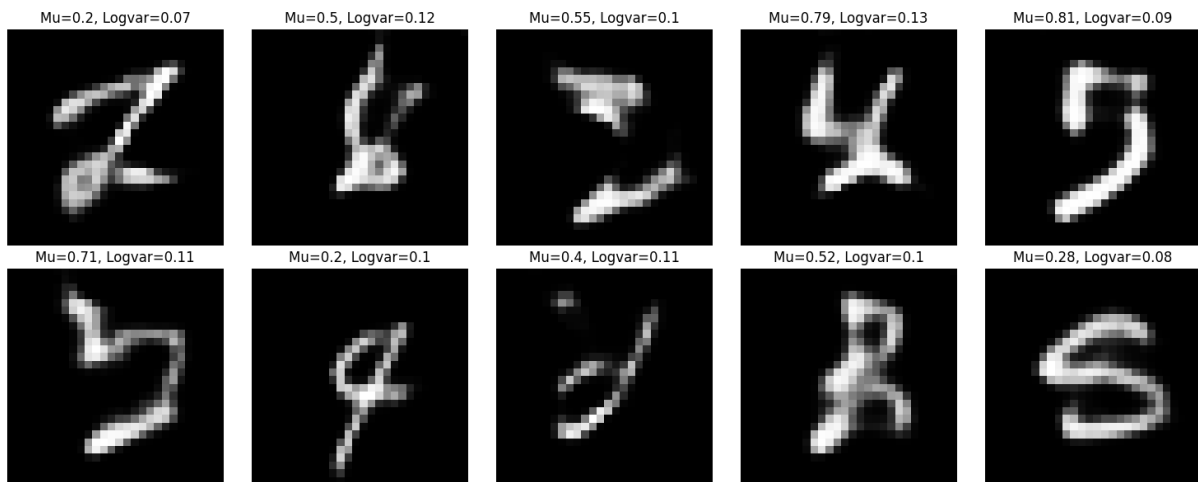


Figura 2: Resultados de generación de dígitos a partir de muestreo de distribución normal en autoencoder variacional convolucional, entrenada con la base de datos MNIST.